Progetto di Laboratorio di Sistemi Operativi — Traccia B

Autore

Nome e cognome	Matricola
Francesco Piscopo	N86005360

Indice

1. Introduzione
2. Guida d'uso
   - 2.1 Struttura del progetto
   - 2.2 Compilazione
   - 2.3 Uso del server
   - 2.4 Uso del client
   - 2.5 Esecuzione con Docker
   - 2.6 Esecuzione senza Docker
3. Protocollo applicativo
   - 3.1 Caratteristiche generali
   - 3.2 Fase di connessione e autenticazione
   - 3.3 Fase di gioco
   - 3.4 Riepilogo dei messaggi
4. Dettagli implementativi
   - 4.1 Strutture dati del server
   - 4.2 Generazione della mappa
   - 4.3 Architettura concorrente del server
   - 4.4 Sincronizzazione: mutex e variabili di condizione
   - 4.5 Timeout globale e calcolo del vincitore
   - 4.6 Invio periodico della mappa globale con `select`
   - 4.7 Fog-of-war e mappa locale
   - 4.8 Registrazione e persistenza degli utenti
   - 4.9 I/O affidabile su socket
   - 4.10 Il client
   - 4.11 Terminazione ordinata del server
5. Limitazioni note e possibili estensioni

1. Introduzione

Il progetto realizza un sistema client–server in linguaggio C su piattaforma UNIX/Linux, in cui più giocatori si contendono le celle di una mappa a griglia. Le regole sono quelle della Traccia B:

- la mappa è una griglia di celle che possono essere **traversabili** oppure **celle muro**;
- quando un giocatore entra in una cella libera, questa diventa **sua**;
- i muri non sono noti in anticipo: ogni giocatore li **scopre esplorando** (con una meccanica *fog-of-war*), mentre la proprietà delle celle è pubblica;
- la partita termina allo scadere di un **timeout globale**, vince chi possiede più celle;
- per giocare occorre **registrarsi** (nickname e password); non c'è un limite a priori al numero di utenti.

Il server è **multithread** (un thread per client, più un thread che gestisce il timer della partita) e comunica con i client tramite **socket TCP**. Il client usa invece `select` per attendere contemporaneamente i comandi dell'utente e i messaggi del server. Entrambi i programmi usano soltanto system call contenute all'interno delle librerie standard C.

---

2. Guida d'uso

2.1 Struttura del progetto

File più importanti:

File	Descrizione
server/server.c	Sorgente del server
client/client.c	Sorgente del client
user.data	File contenente nome e password degli utenti registrati
compose.yaml	File docker compose
cmd.txt	Comandi Docker utili - (contenuti anche in questa relazione)

Altri file di cui tenere nota:

File	Descrizione
TracciaB	Traccia del progetto

File	Descrizione
client.c	Sorgente del client in root
server.c	Sorgente del server in root
makefile	File Cmake per realizzare test sulla stessa macchina senza docker - (questo usa i sorgenti nella root)

2.2 Compilazione

Il server usa i thread POSIX, quindi richiede il flag `-pthread`. Il client non ne ha bisogno.

```
# Server
gcc -Wall -Wextra -pthread -o server.elf server/server.c

# Client
gcc -Wall -Wextra -o client.elf client/client.c
```

Entrambi i sorgenti definiscono `_GNU_SOURCE` per avere accesso ad alcune delle funzioni e usano solo header POSIX/libc (`sys/socket.h`, `netinet/in.h`, `netdb.h`, `pthread.h`, ...).

2.3 Uso del server

```
./server.elf <durata>
```

Parametro	Significato
`<durata>`	Durata della partita in secondi, al suo scadere la partita termina.

Esempio: `./server.elf 120` avvia una partita da due minuti.

Note di funzionamento:

- il server ascolta su **tutte le interfacce** (INADDR_ANY), scelta basata sul fatto che il client concettualmente si connetterà all'ip pubblico e non ad un ip locale specifico;
- il server ascolta sulla **porta TCP 5050** (costante definita nel codice);
- all'avvio legge il file 'user.data' e carica gli utenti già registrati; se il file non esiste ne verrà creato uno alla prima registrazione;
- il server non legge nulla da **stdin** e interagisce con **stdout** soltanto per facilitare il debugging, ma non sono necessarie
- il server termina da solo quando la partita è finita.

Se i parametri sono sbagliati il server stampa un messaggio di errore e termina con codice 1.

2.4 Uso del client

```
./client.elf <nome-o-indirizzo-IP-del-server> <porta>
```

Esempi:

```
./client.elf 192.168.1.20 5050
./client.elf localhost 5050
```

Dopo la connessione il client richiede nickname e password. Se il nickname non è mai stato registrato, viene creato un nuovo utente con la password indicata; se invece esiste, la password deve coincidere. Nickname e password sono al massimo di 15 caratteri e non possono contenere spazi. Lo stesso utente non può essere collegato due volte contemporaneamente.

Una volta in gioco, i comandi (da confermare con Invio) sono:

Comando	Effetto
`w` / `W`	Muove il giocatore verso l'alto (riga − 1)
`s` / `S`	Muove il giocatore verso il basso (riga + 1)
`a` / `A`	Muove il giocatore verso sinistra (colonna − 1)
`d` / `D`	Muove il giocatore verso destra (colonna + 1)
`q` / `Q`	Disconnette il giocatore e chiude il client

L'output a terminale comprende:

- mappa locale, stampata dopo ogni mossa. Legenda: `#` cella sconosciuta, `X` muro, `0` cella libera, `@` posizione del giocatore;
- mappa globale della proprietà, stampata ogni 5 secondi. Legenda: `.` cella non conquistata, `0`–`9` identificativo del proprietario, `*` proprietario con id ≥ 10;
- elenco dei giocatori collegati, stampato insieme alla mappa globale, con nickname, id e coordinate indicate da una coppia riga-colonna;
- punteggio del giocatore;
- risultato della partita.

2.5 Esecuzione con Docker

Per facilitare i test su più "macchine" il progetto include un file docker compose, *compose.yaml*, con due servizi, **server** e **client**, questi sono collegati da una rete Docker dedicata (miarete).
Il server pubblica la porta 5050 sull'host e monta il file user.data nel container usando i *volumi docker*, così che le registrazioni siano persistenti.

Di seguito sono elencati i comandi da eseguire per il corretto funzionamento del docker compose (contenuti anche in *cmd.txt*):

```
# 1. creare la rete (una sola volta)
sudo docker network create miarete

# 2. costruire e avviare il server in background, d sta per detach mode
sudo docker compose up --build -d server

# 3. avviare uno o più client con terminale interattivo, rm rimuove il
container una volta finito
sudo docker compose run --rm -it client

# 4. pulizia dei container terminati
sudo docker container prune
```

Nel servizio client il comando è './client.elf server 5050': all'interno della rete Docker il server è raggiungibile con il nome del servizio (server), risolto da getaddrinfo.
Il server viene avviato con './server.elf 120', quindi la partita dura 120 secondi.

2.6 Esecuzione senza Docker

Per eseguire un test sulla stessa macchina, senza utilizzo di docker e quindi con ip locali, basta eseguire i seguenti comandi:

```
# 1. eseguire il makefile
make

# 2A. su un terminale eseguire un comando server
./server.elf 120

# 2B. su altri terminali [1,N] eseguire un comando client con indirizzo
localhost
./client.elf localhost 5050

# 3. eseguire una pulizia degli eseguibili
make clean
```

---

3. Protocollo applicativo

3.1 Caratteristiche generali

Il protocollo è di tipologia **testuale**:

- ogni messaggio è composto da una o più **righe** terminate da '\n';
- i campi di una riga sono separati da **un singolo spazio**;
- i messaggi composti (mappa locale, mappa globale) sono **blocchi** formati da un'intestazione, un numero noto di righe di dati e una riga terminatrice 'END';
- la comunicazione avviene su un'unica connessione TCP per giocatore, che rimane aperta per tutta la sessione;

Poiché TCP è uno *stream*, sia il client sia il server leggono dalla socket **un byte alla volta** fino al '\n' (funzione *receiveLine*), ricostruendo così le righe.
La dimensione della mappa è una costante (SIZE = 20). Il server la comunica comunque nelle intestazioni dei blocchi, e il client verifica che coincida con la propria.

3.2 Fase di connessione e autenticazione

1. Il client si connette (via connect());
2. Il server accetta la connessione e invia ready;
3. Il server invia 'Ready'. Il client attende questa riga al massimo 5 secondi (timeout di ricezione SO_RCVTIMEO); se non arriva, considera il server non disponibile.
4. Il client invia due righe: nickname e password.
5. Il server risponde con 'Success' o 'Error'.
   - Nickname nuovo > 'Success' ed inoltre l'utente viene registrato (e salvato su file).
   - Nickname esistente, password corretta > 'Success'.
   - Password errata, campi vuoti o contenenti spazi, partita già terminata, errore di scrittura → 'Error' e chiusura della connessione.
6. Dopo 'Success' il server sceglie una cella di partenza libera e casuale e invia una seconda riga 'Ready', seguita dal primo blocco 'LOCAL' e dal primo blocco 'GLOBAL'.

3.3 Fase di gioco

Durante la partita il traffico è **asincrono** in entrambe le direzioni:

- il **client** invia comandi di movimento quando l'utente li digita;
- il **server** risponde a ogni comando e, indipendentemente dai comandi, invia la mappa globale **ogni T = 5 secondi**.

Blocco `LOCAL` - mappa locale

Inviato dopo il login e dopo ogni comando del client al server

```
LOCAL 20
MAPROW ####################
MAPROW ####################
MAPROW ###XOX##############
MAPROW ###0@0##############
...
END
```

- 'LOCAL <N>': intestazione con la dimensione 'N' della mappa;
- 'MAPROW <N caratteri>': una riga della mappa, con codifica nominata precedentemente;
- 'END': fine mappa

La mappa locale contiene esclusivamente le celle che il giocatore ha scoperto

Blocco `GLOBAL` - mappa globale

Inviato dopo il login e poi ogni 5 secondi

```
GLOBAL 20 2
OWNROW ....................
OWNROW .........1..........
...
PLAYER 0 3 4 mario
PLAYER 1 7 12 luca
END
```

- 'GLOBAL <N> <P>': dimensione della mappa e numero di giocatori collegati;
- 'OWNROW <N caratteri>': celle libere e conquistate;
- 'PLAYER <id> <riga> <colonna> <nickname>': un record per ogni giocatore connesso;
- 'END': fine mappa

Messaggio GAMEOVER

Messaggio inviato alla fine del timeout della partita ed inviato ad ogni giocatore, dopo questo il server chiude le connessioni

```
GAMEOVER <idVincitore> <celleVincitore> <celleGiocatore> <nomeVincitore>
```

Campo	Significato
idVincitore	id del vincitore, oppure -1 in caso di pareggio
celleVincitore	numero di celle conquistate dal vincitore
celleGiocatore	numero di celle conquistate dal giocatore locale
nomeVincitore	nickname del vincitore

3.4 Riepilogo dei messaggi

Direzione	Messaggio	Quando
S -> C	Ready	Dopo la connessione; dopo il login
C -> S	nickname, password	Subito dopo il primo Ready
S -> C	Success / Error	Esito dell'autenticazione
S -> C	LOCAL ... END	Dopo il login e dopo ogni comando
S -> C	GLOBAL ... END	Dopo il login e ogni 5 s
C -> S	w/W, a/A, s/S, d/D	Movimento
S -> C	Moved x / Blocked x	Risposta a un comando
S -> C	GAMEOVER ...	Allo scadere del timer globale
C -> S	q/Q	Disconnessione

---

4. Dettagli implementativi

4.1 Strutture dati del server

Tutto lo stato di gioco è contenuto in due strutture, allocate sullo stack del main e condivise per puntatore con i thread.

```
struct map {
  int map[SIZE][SIZE];        // WALL (0) o PATH (1): mappa completa degli ostacoli
  int ownerShip[SIZE][SIZE];  // id del proprietario, -1 se la cella non è di nessuno
};

struct user {
  int  id;
```

```
  int  x, y;                  // posizione corrente
  char name[16];
  char password[16];
  int  score;
  int  localMap[SIZE][SIZE];  // celle scoperte da questo giocatore (-1 =
ignota)
  int  isConnected;
};
```

Corrispondono a quanto richiesto dalla traccia: il server mantiene la **mappa completa degli ostacoli** (map.map), la **mappa globale della proprietà** (map.ownerShip) e, per ogni giocatore, **posizione** e **celle muro scoperte** (localMap). Gli utenti sono contenuti in un array 'User u[1024]'; l'indice coincide con l'id, che è anche il valore scritto nella mappa di proprietà.

4.2 Generazione della mappa

La mappa viene generata una volta sola, all'avvio, in due passi:

```
int generateRandomMap(Map* mapOut) {
  for (int i = 0; i < SIZE; i++)
    for (int j = 0; j < SIZE; j++)
      mapOut->map[i][j] = (rand() % 2 == 0) ? WALL : PATH;   // 50% muro,
50% libera
  return 0;
}
```

Il generatore è inizializzato con 'srand(time(NULL))', quindi ogni esecuzione produce un territorio diverso. Il secondo passo, 'cleanUpMap', trasforma in muro le celle libere circondate da muri su tutti e quattro i lati: sono celle irraggiungibili, e sarebbe inutile (e potenzialmente pericoloso) usarle come punto di partenza, e' importante notare pero' come questo non sia un sistema infallibile (si possono infatti chreare chiazze chiuse di celle traversabili).

4.3 Architettura concorrente del server

Il requisito principale è gestire più client contemporaneamente, accettando nuove connessioni e inviando aggiornamenti periodici **senza mai bloccarsi** su un singolo client. La soluzione adottata è un modello a thread:

Thread	Compito
main	Ciclo di accept; crea un thread per ogni connessione; a fine partita attende la disconnessione di tutti e fa il join del thread di gioco

Thread	Compito
thread di gioco (runGame)	Attende lo scadere del timeout globale, poi calcola punteggi e vincitore
thread client (handleClient, uno per connessione)	Autenticazione, ricezione comandi, invio di mappa locale/globale

I thread dei client sono creati in modalità **detached** (PTHREAD_CREATE_DETACHED): nessuno ne attende la terminazione, quindi le loro risorse vengono rilasciate automaticamente all'uscita, senza pericolo di zombie.

Il ciclo di accept non è bloccante in modo indefinito: usa select con un timeout sulla socket di ascolto, così da poter controllare periodicamente se la partita è finita per smettere di accettare connessioni.

```
while (1) {
  pthread_mutex_lock(&usersMutex);
  int finished = gameOver;
  pthread_mutex_unlock(&usersMutex);
  if (finished) break;

  fd_set readSet;
  FD_ZERO(&readSet);
  FD_SET(serverSocket, &readSet);
  struct timeval timeout = {.tv_sec = 0, .tv_usec = 250000};
  int ready = select(serverSocket + 1, &readSet, NULL, NULL, &timeout);
  if (ready < 0) { if (errno == EINTR) continue; perror("select server
socket"); break; }
  if (ready == 0) continue;

  int clientSocket = accept(serverSocket, NULL, NULL);
  ...
  pthread_create(&clientThread, &clientThreadAttributes, handleClient,
clientArgs);
}
```

La socket di ascolto usa **SO_REUSEADDR**, in modo da non avere problemi dovuti a delle bind troppo rapide da parte del server dopo un riavvio.

4.4 Sincronizzazione

La sincronizzazione per gestire la memoria condivisa in questo progetto avviene tramite l'utilizzo di **mutex** e **variabili di condizione**;

I dati condivisi (mappa di proprietà, posizioni, punteggi, flag 'gameOver', contatore 'activePlayers') sono protetti da un unico mutex globale, **usersMutex**. Questa scelta evita problemi di deadlock da ordinamento dei lock e, con sezioni critiche brevissime, non costituisce un *bottleneck*.

Per evitare di tenere il lock durante operazioni lente di rete, il server segue sempre lo schema **copia sotto lock**. Ad esempio in 'sendGlobalInfo':

```
pthread_mutex_lock(&usersMutex);
memcpy(ownerShip, map->ownerShip, sizeof(ownerShip));     // snapshot della
proprietà
for (int i = 0; i < id; i++) {                            // snapshot dei
giocatori connessi
  if (!users[i].isConnected) continue;
  players[playerCount].id = users[i].id;
  players[playerCount].x  = users[i].x;
  players[playerCount].y  = users[i].y;
  memcpy(players[playerCount].name, users[i].name,
sizeof(players[playerCount].name));
  playerCount++;
}
pthread_mutex_unlock(&usersMutex);
/* ... ora si formattano e si inviano le righe senza alcun lock ... */
```

In questo modo un client lento o bloccato in 'send' non ferma gli altri giocatori.

Il server usa inoltre due **variabili di condizione** associate allo stesso mutex:

Variabile	Chi aspetta	Chi segnala	Scopo
gameFinishedCondition	thread di gioco, con pthread_cond_timedwait	thread di gioco al termine	Attesa del timeout globale
activePlayersCondition	thread main, a fine partita	disconnectPlayer ad ogni disconnessione	Il main sa quando l'ultimo giocatore se ne è andato

Il movimento è eseguito interamente sotto lock, perché modifica sia la posizione del giocatore sia la mappa di proprietà:

```
int movePlayer(User* user, const char* command, Map* map) {
  int x = user->x, y = user->y;
  switch (command[0]) {
    case 'w': case 'W': x--; break;
    case 's': case 'S': x++; break;
    case 'a': case 'A': y--; break;
    case 'd': case 'D': y++; break;
    default: return 0;
  }
  if (command[1] != '\0' || x < 0 || x >= SIZE || y < 0 || y >= SIZE ||
      map->map[x][y] != PATH) return 0;          // fuori mappa, muro o
comando malformato

  user->x = x; user->y = y;
  map->ownerShip[x][y] = user->id;               // la cella passa a questo
giocatore
  return 1;
}
```

La regola "ogni cella appartiene all'ultimo giocatore che vi è passato" è realizzata semplicemente **sovrascrivendo** `ownerShip[x][y]` con l'id del giocatore: la conquista viene così ribaltata in modo naturale quando un altro giocatore ripassa sulla stessa cella. Il posizionamento iniziale (`placePlayer`) sceglie una cella libera a partire da un indice casuale, scartando quelle occupate da un altro giocatore connesso, e conquista subito la cella di partenza.

4.5 Timeout globale e calcolo del vincitore

Il timer di gioco è delegato a un thread dedicato che calcola una **scadenza assoluta** ('CLOCK_REALTIME + gameLength') e si mette in attesa sulla variabile di condizione con 'pthread_cond_timedwait':

```
deadline.tv_sec += gameLength;

pthread_mutex_lock(&usersMutex);
while (!gameOver) {
  int waitResult = pthread_cond_timedwait(&gameFinishedCondition,
&usersMutex, &deadline);
  if (waitResult == ETIMEDOUT) break;      // il tempo è scaduto
  if (waitResult != 0) {
    char buf2[]="Loaded user: ";
    write(STDERR_FILENO, buf2, sizeof(buf2));
    write(STDERR_FILENO, strerror(waitResult),
strlen(strerror(waitResult)));
```

```
    write(STDERR_FILENO, &newLine, sizeof(char));
    break;
  }
}
```

Dopo il break post-attesa, il thread:

1. azzera i punteggi e li ricalcola scorrendo una sola volta la mappa di proprietà (un punto per ogni cella posseduta);
2. determina il vincitore, o il pareggio(winnerId = -1);
3. imposta gameOver=1 ed esegue pthread_cond_broadcast.

```
for (int i = 0; i < id; i++) {
    if (users[i].score > winnerScore) {
      winnerId = i;
      winnerScore = users[i].score;
      tied = 0;
    } else if (users[i].score == winnerScore && users[i].score > 0) {
      tied = 1;
    }
}
if (tied) winnerId = -1;
gameOver = 1;
pthread_cond_broadcast(&gameFinishedCondition);
pthread_mutex_unlock(&usersMutex);
```

I thread dei client controllano 'gameOver' ad ogni iterazione del loro ciclo e, se il flag vale allora inviano GAMEOVER e chiudono la connessione.

4.6 Invio periodico della mappa globale con `select`

Ogni thread client ha due compiti: **ricevere comandi** (bloccante) e **inviare la mappa globale ogni 5 secondi** (periodico). Per evitare di usare due thread per client, si usa **select** con un timeout calcolato sul tempo restante alla prossima scadenza: se il client invia un comando prima, select ritorna con la socket pronta; altrimenti ritorna con 0 al momento giusto per inviare l'aggiornamento.

```
    //get current time
    struct timespec now;
    if (clock_gettime(CLOCK_MONOTONIC, &now) < 0) {
      perror("clock_gettime");
      break;
    }
```

```
    //if the map should have been sent already -> send it and set the next
send time
    if(now.tv_sec > nextMapSend.tv_sec || (now.tv_sec == nextMapSend.tv_sec
&& now.tv_nsec >= nextMapSend.tv_nsec)){
      if(sendGlobalInfo(clientSocket, users, user, map)<0){
        if(errno != EPIPE && errno != ECONNRESET) perror("send global
info");
        break;
      }
      if(clock_gettime(CLOCK_MONOTONIC, &nextMapSend)<0){
        perror("clock_gettime");
        break;
      }
      nextMapSend.tv_sec += 5;
      continue;
    }

    //create a timeout window for the select, and convert different size
values (nano to micro div by 1000)
    struct timeval timeout = {0};
    timeout.tv_sec = nextMapSend.tv_sec - now.tv_sec;
    if(nextMapSend.tv_nsec >= now.tv_nsec){
      timeout.tv_usec = (nextMapSend.tv_nsec - now.tv_nsec) / 1000;
    } else{
      timeout.tv_sec--;
      timeout.tv_usec = 1000000 + (nextMapSend.tv_nsec - now.tv_nsec) /
1000;
    }

    //create select
    fd_set readSet;
    FD_ZERO(&readSet);
    FD_SET(clientSocket, &readSet);
    int ready = select(clientSocket + 1, &readSet, NULL, NULL, &timeout);
    //error
    if(ready < 0){
      if (errno == EINTR) continue;
      perror("select client socket");
      break;
    }
    //select is ready therefore 5 seconds have passed, send global info
    if (ready == 0) {
      if (sendGlobalInfo(clientSocket, users, user, map) < 0) {
        if (errno != EPIPE && errno != ECONNRESET) perror("send global
info");
```

```
        break;
      }
      if (clock_gettime(CLOCK_MONOTONIC, &nextMapSend) < 0) {
        perror("clock_gettime");
        break;
      }
      nextMapSend.tv_sec += 5;
      continue;
    }

    //select is not ready, therefore we should try reading the command from
the client
    lineResult = receiveLine(clientSocket, buffer, sizeof(buffer));
    if (lineResult == 0) break;
    if (lineResult < 0) {
      if (errno != 0) perror("recv client command");
      break;
    }
```

Due scelte interessanti:

- CLOCK_MONOTONIC viene usato come tipologia di orologio che avanza in maniera costante indipendentemente da eventuali alterazioni temporali di sistema (ref: https://www.sourceware.org/glibc/manual/2.33/html_node/Getting-the-Time.html);
- l'intervallo viene **ricalcolato dopo ogni invio**, e se la scadenza è già passata prima ancora di chiamare select l'invio avviene subito, quindi un client con input continui lo stesso non interrompe il flusso delle operazioni.

4.7 Fog-of-war e mappa locale

Il fog-of-war è realizzato tenendo, per ogni utente, una matrice 'localMap' inizializzata a -1 (cella sconosciuta). Ad ogni invio della mappa locale il server chiama *revealAround*, che copia dalla mappa reale alla matrice dell'utente le celle del quadrato 3×3 centrato sulla sua posizione (la cella corrente, le quattro adiacenti e le quattro diagonali):

```
void revealAround(User* u, const Map* m) {
  for (int dx = -1; dx <= 1; dx++)
    for (int dy = -1; dy <= 1; dy++) {
      int nx = u->x + dx, ny = u->y + dy;
      if (nx >= 0 && nx < SIZE && ny >= 0 && ny < SIZE)
        u->localMap[nx][ny] = m->map[nx][ny];
    }
}
```

Reputo interessante notare questo codice in quanto l'approccio banale (che coincide con quello antecedente questo) sarebbe realizzare una serie di if-else con dei flag per poter gestire i casi in cui ci troviamo al limite della mappa, questa implementazione invece è estremamente elegante, efficiente e leggibile.

Il giocatore ricorda tutto ciò che ha esplorato e riceve solamente le informazioni scoperte in quanto il server non manda mai l'intera mappa.

4.8 Registrazione e persistenza degli utenti

Gli utenti sono memorizzati nel file 'user.data', una riga per utente nel formato 'nickname password'.
All'avvio la funzione *loadUsers* legge il file un carattere alla volta con read(), usando lo spazio come separatore fra i campi e il newline come fine record, e popola l'array degli utenti.
Va nominata una importante limitazione, ovvero che data questa implementazione se il file 'user.data' viene alterato da un programma terzo o semplicemente a mano, si potrebbe rompere la sintassi e rendere non funzionante questa funzione (un errore semplice potrebbe essere quello di non inserire una newline per l'ultima riga presente nel file (ovvero l'ultimo utente)).

authenticateOrRegister implementa in un'unica funzione il **login e la registrazione** sotto usersMutex; Da notare:

1. i campi vuoti o contenenti spazi vengono rifiutati (lo spazio romperebbe il formato del file e del protocollo);
2. se il nickname esiste, si confronta la password e si verifica che l'utente non sia già connesso (impedisce la doppia sessione);
3. se non esiste il file user.data viene creato con permessi 0600 ovvero scrittura e lettura solo per l'autore, se no si scrive alla fine del file.

Grazie alla persistenza su file, un utente registrato in una sessione può rientrare dopo il riavvio del server. Con Docker il file è montato come **volume** ('./user.data:/dir/user.data') per sopravvivere alla distruzione del container.

4.9 I/O affidabile su socket

Le chiamate send e recv possono trasferire meno byte del richiesto e possono essere interrotte da segnali. Le ho incapsulate in funzioni di supporto:
La prima: **sendAll**.

```
int sendAll(int socket, const char* buffer, size_t length) {
  size_t sent = 0;
  while (sent < length) {
```

```
    ssize_t result = send(socket, buffer + sent, length - sent,
MSG_NOSIGNAL);
    if (result < 0) { if (errno == EINTR) continue; return -1; }
    if (result == 0) return -1;
    sent += (size_t)result;
  }
  return 0;
}
```

Le righe di mappa vengono inviate costruendo il buffer in memoria e spedendolo con una sola sendAll.
Di seguito alcune note riguardo la funzione:

- sendAll ripete la send finché tutti i byte non sono partiti e riprova in caso di EINTR;
- il flag MSG_NOSIGNALevita che la scrittura su una connessione chiusa dal client generi SIGPIPE, che terminerebbe l'intero server: l'errore viene invece restituito e gestito disconnettendo il solo giocatore interessato; (ref: https://linux.die.net/man/2/send)

La seconda: **receiveLine**:

```
int receiveLine(int socket, char* buffer, size_t capacity) {
  size_t length = 0;
  while (1) {
    char ch;
    ssize_t received = recv(socket, &ch, 1, 0);
    if (received < 0) {
      if (errno == EINTR) continue;
      return -1;
    }
    if (received == 0) return length == 0 ? 0 : -1;
    if (ch == '\n') {
      buffer[length] = '\0';
      return 1;
    }
    if (ch != '\r') {
      if (length + 1 >= capacity) return -1;
      buffer[length++] = ch;
    }
  }
}
```

- receiveLine legge un byte per volta fino a '\n', ignora i '\r', e rifiuta le righe che superano la capacità del buffer, difendendo il server da buffer overflow; restituisce 0 alla chiusura pulita della connessione e -1 in caso di errore.

4.10 Il client

Il client è un programma a processo singolo che alterna lettura da tastiera e dalla socket grazie a **select**:

```
fd_set readSet;
FD_ZERO(&readSet);
FD_SET(clientSocket, &readSet);
FD_SET(STDIN_FILENO, &readSet);
int ready = select(maxFd + 1, &readSet, NULL, NULL, NULL);   // attesa su entrambe le sorgenti

if (FD_ISSET(clientSocket, &readSet)) { /* parsing messaggio dal server: LOCAL, GLOBAL, GAMEOVER, ... */ }
if (FD_ISSET(STDIN_FILENO, &readSet)) { /* lettura da stdin e send del comando */ }
```

Questo permette di mostrare la mappa globale non appena arriva (ogni 5 secondi) anche mentre l'utente non sta digitando nulla.

Altri aspetti da segnalare:

- **risoluzione dell'indirizzo** con **getaddrinfo**(AF_INET, SOCK_STREAM): l'utente può indicare indifferentemente un nome host o un indirizzo IP;
- **timeout iniziale**: dopo la connect() si imposta SO_RCVTIMEO a 5 secondi per attendere il primo 'Ready', per evitare che il client si connetta senza ricevere risposte; subito dopo il timeout viene rimosso;
- **validazione del protocollo**: ogni blocco ricevuto viene controllato (LOCAL/GLOBAL/MAPROW/OWNROW/PLAYER, variabili varie, presenza di END), e il client termina con un messaggio di errore su *stderr* se il server invia dati non conformi;
- **parsing dei campi numerici con strtol** e controllo del puntatore finale (parseIntField);
- **uscita**: con q/Q o EOF su stdin il client esce dal ciclo e chiude la socket; alla ricezione di GAMEOVER stampa punteggio e vincitore e termina; la chiusura della connessione da parte del server viene rilevata e segnalata come *Server disconnected*.

4.11 Terminazione ordinata del server

Quando il timer scade e gameOver diventa 1, il main esce dal ciclo di accept(), chiude la socket di ascolto e **attende che tutti i giocatori si disconnettano**, usando il contatore activePlayers e la variabile di condizione activePlayersCondition:

```
pthread_mutex_lock(&usersMutex);
while (activePlayers > 0)
```

```
    pthread_cond_wait(&activePlayersCondition, &usersMutex);
pthread_mutex_unlock(&usersMutex);

pthread_join(gameThread, NULL);
return 0;
```

Ogni disconnectPlayer decrementa il contatore e fa pthread_cond_broadcast, e poiché la chiamata a pthread_cond_wait è dentro un while che ricontrolla la condizione, eventuali risvegli spuri sono innocui. In questo modo i thread detached dei client riescono a inviare GAMEOVER a tutti prima che il processo termini; il thread di gioco, che ha già concluso, viene raccolto con pthread_join.

---

5. Limitazioni note e possibili estensioni

Per completezza si riportano alcune scelte semplificative del prototipo:

- **Parametri fissi**: dimensione della mappa (20×20), porta (5050), periodo di aggiornamento globale (5 s) e raggio di scoperta (3×3) sono costanti nel codice; solo la durata della partita è un parametro da riga di comando. Renderli configurabili è un'estensione immediata.
- **Connettività della mappa**: la generazione casuale elimina solo le celle libere totalmente isolate; non garantisce che tutte le celle libere siano raggiungibili tra loro.
- **Sicurezza**: le password sono memorizzate e trasmesse in chiaro. Per un uso reale sarebbero necessari hashing con sale e cifratura del canale.
- **Limiti dimensionali**: l'archivio è limitato a 1024 utenti, e nickname e password a 15 caratteri.